

Análise Semântica

Visão Geral

A análise semântica percorre a AST produzida pelo parser e verifica a correcção semântica do programa antes da tradução. É implementada no `SemanticAnalyser`, que herda de `ASTVisitor` e usa uma `SymbolTable` para gerir a informação sobre variáveis e subprogramas.

```
AST (do parser)
  ↓
[SemanticAnalyser]
  ├── SymbolTable (registra e resolve símbolos)
  ├── Verificação de tipos
  ├── Validação de labels (GOTO / DO)
  └── Protecção de variáveis de controlo DO
  ↓
list[SemanticError] (vazia se sem erros)
```

Tabela de Símbolos (`symbolTable.py`)

`VarSymbol` — entrada para variáveis

```
@dataclass
class VarSymbol:
    name: str
    var_type: str # 'INTEGER' | 'REAL' | 'LOGICAL' | 'CHARACTER'
    | 'UNKNOWN'
    dimension: int = 0 # 0 = escalar; >= 1 = array
    initialized: bool = False
    index: int = -1 # slot na stack/frame da VM
    is_param: bool = False
```

Campo `index`: atribuído automaticamente pela tabela de símbolos no momento da declaração. Para variáveis locais, é um inteiro não-negativo (posição no frame local). Para parâmetros formais, é negativo (ver convenção de chamada abaixo). O valor `-1` indica que o símbolo ainda não foi registado.

Campo `dimension`: `0` para escalares; para arrays (declarados com `INTEGER A(N)`), armazena a dimensão `N`. Arrays são sempre unidimensionais nesta implementação.

Propriedade `size`: sempre devolve `1`. Um array ocupa apenas 1 slot na stack da VM, pois esse slot guarda o endereço do bloco alocado na heap via `ALLOC`. Os elementos do array vivem na heap, não no frame.

Campo `initialized`: rastreado apenas para a variável de retorno de funções (ver verificação de inicialização abaixo). Para variáveis regulares, a análise não faz data-flow completo — não detecta uso de variável não inicializada em geral, apenas o caso específico da função.

SubprogramSymbol — entrada para funções e subrotinas

```
@dataclass
class SubprogramSymbol:
    name: str
    kind: str # 'FUNCTION' | 'SUBROUTINE'
    return_type: str | None # None para subrotinas
    param_names: list[str]
    param_types: list[str]
```

O `return_type` de uma função pode ser `'UNKNOWN'` se não for possível determiná-lo a partir das declarações (tipo implícito não suportado). A aridade (`arity`) é calculada a partir do comprimento de `param_names`.

Estrutura de scopes

```
class SymbolTable:
    def __init__(self):
        self._table: list[dict] = [{}] # scope global (índice 0)
        self._counters: list[int] = [0] # contador de slots por scope
```

A tabela é uma lista de dicionários, onde `_table[0]` é sempre o scope global. Cada `push()` adiciona um novo scope para a unidade de programa em análise; cada `pop()` remove-o.

Método	Descrição
<code>push()</code>	Entra num novo scope (chamado ao iniciar cada função/subrotina/programa)
<code>pop()</code>	Sai do scope actual (chamado no final de cada unidade)
<code>declare_var(sym)</code>	Regista variável no scope actual; atribui <code>index = counter++</code>
<code>declare_param(sym, i)</code>	Regista parâmetro com <code>index = -(i+1)</code> (negativo)
<code>declare_subprogram(sym)</code>	Regista sempre no scope global (<code>_table[0]</code>)
<code>lookup_var(name)</code>	Pesquisa do scope mais interior para o exterior
<code>lookup_subprogram(name)</code>	Pesquisa apenas no scope global
<code>initialize(name)</code>	Marca variável como inicializada

Convenção de índices de parâmetros

Os parâmetros formais recebem índices negativos:

- Parâmetro 0 → `index = -1`
- Parâmetro 1 → `index = -2`
- Parâmetro N → `index = -(N+1)`

Esta convenção reflecte directamente o layout do stack frame da EWVM: `fp[-1]` é o argumento mais próximo do topo (o último passado), `fp[-2]` o segundo, etc. A variável de retorno de uma função ocupa `fp[0]` (primeiro slot local, index 0).

SemanticAnalyser

Estado interno

```
_errors:          list[SemanticError]    # erros acumulados
_symbol_table:    SymbolTable             # tabela de símbolos activa
_labels:          dict[int, int]          # {label → linha de definição} na
unidade actual
_goto_refs:       list[tuple[int, int]]   # [(label, linha)] referências GOTO/DO
a validar
_do_stack:        list[tuple[int, str]]   # [(label_term, var_ctrl)] loops DO
activos
_do_vars:         set[str]                # nomes de variáveis de controlo
activas
_current_unit:    str | None              # nome da unidade em análise
_current_kind:    str | None              # 'PROGRAM' | 'FUNCTION' |
'SUBROUTINE'
```

Fases da Análise

Fase 1: Pré-registo de subprogramas

```
def visit_Program(self, node: Program) -> None:
    for unit in node.units:
        if isinstance(unit, (FunctionDef, SubroutineDef)):
            self._regist_subprogram(unit)
    for unit in node.units:
        self.visit(unit)
```

Todos os subprogramas são registados no scope global **antes** de qualquer análise. Isto permite que um subprograma seja chamado antes de ser definido no ficheiro-fonte — comportamento normal em Fortran 77 com múltiplas unidades de programa.

Para determinar os tipos dos parâmetros e o tipo de retorno no momento do registo, `_regist_subprogram` analisa as declarações de tipo no body do subprograma sem criar scope. Tipos não determinados ficam como `'UNKNOWN'`.

Fase 2: Análise de cada unidade

Para cada unidade (programa principal, função, ou subrotina):

1. `_reset_labels()` — limpa os dicionários de labels da unidade anterior.

2. `_symbol_table.push()` — cria um novo scope.
3. Parâmetros formais são declarados com `declare_param()`.
4. Para funções: a variável de retorno (com o mesmo nome da função) é declarada como variável local com índice 0.
5. O body é visitado (declarações → instruções).
6. `_check_label_refs()` — valida todas as referências a labels acumuladas.
7. `_symbol_table.pop()` — remove o scope.

Fase 3: Validação de labels

No final de cada unidade, `_check_label_refs()` verifica que todos os labels referenciados por **GOTO** e **DO** estão definidos. Os labels são locais a cada unidade de programa.

Verificações Semânticas

Declarações (`visit_TypeDecl`)

- **Re-declaração:** erro se a variável já estiver declarada no scope actual com tipo diferente de `'UNKNOWN'`.
- **Refinamento de parâmetros:** se a variável já existe com tipo `'UNKNOWN'` (é um parâmetro formal sem tipo), actualiza o tipo e a dimensão sem alterar o índice.

Atribuições (`visit_Assign`)

1. **Protecção de variável de controlo DO:** erro se o alvo da atribuição for uma variável de controlo de um DO loop activo.
2. **Compatibilidade de tipos:** chama `_check_assign_compat()`.
3. **Marcação como inicializada:** chama `_symbol_table.initialize()` no alvo.

Compatibilidade de tipos em atribuições

```
def _check_assign_compat(self, ltype, rtype, lineno):
    if ltype in _NUMERIC_TYPES and rtype in _NUMERIC_TYPES:
        return # conversão numérica implícita permitida (INTEGER ↔ REAL)
    if ltype != rtype:
        error(...) # tipos incompatíveis
```

A conversão implícita entre **INTEGER** e **REAL** é permitida (como em Fortran 77), mas misturar tipos numéricos com **LOGICAL** ou **CHARACTER** é erro.

Condições IF (`visit_IfThen`, `visit_LogicalIf`)

A condição deve ser do tipo **LOGICAL**. Se o tipo inferido for diferente de **LOGICAL**, é gerado um erro.

DO loops (`visit_DoLoop`)

1. A variável de controlo deve estar declarada e ser **INTEGER** ou **REAL**.
2. Os limites (start, stop) devem ser numéricos.

3. O passo (step, se presente) deve ser numérico.
4. A variável de controlo é adicionada a `_do_vars` durante a análise do body, impedindo modificações.
5. No final do loop, a variável é removida de `_do_vars` e o label do DO é definido.

GOTO (`visit_Goto`)

Regista a referência ao label para validação posterior em `_check_label_refs()`.

Chamadas a subrotinas (`visit_CallStmt`)

1. A subrotina deve estar definida.
2. Não pode chamar uma `FUNCTION` com `CALL`.
3. O número de argumentos deve coincidir com a aridade.
4. Os tipos dos argumentos são verificados se o tipo esperado for conhecido.

Inicialização da variável de retorno (`visit_FunctionDef`)

No final da análise de uma função, verifica se a variável de retorno (o nome da função) foi marcada como inicializada. Se não foi, é um aviso/erro — a função pode não retornar valor.

Acesso a arrays e chamadas de função (`_type_of_var_or_func`)

O nó `VarOrFuncCall` é desambiguado aqui:

- **Caso A** — array local: `var_sym.is_array == True` → verifica que há exactamente 1 índice e que o índice é `INTEGER`.
- **Caso B** — função de utilizador: `func_sym` existe na tabela global → verifica aridade e tipos dos argumentos.
- **Caso C** — escalar indexado: `var_sym` existe mas `is_array == False` → erro.
- **Caso D** — subrotina usada como expressão: `func_sym.is_subroutine == True` → erro.

Inferência de Tipos

O método `_type_of(node)` devolve uma `str` com o tipo da expressão ou `None` se não for determinável (ex: variável não declarada).

Nó	Tipo devolvido
<code>IntLit</code>	'INTEGER'
<code>RealLit</code>	'REAL'
<code>StringLit</code>	'CHARACTER'
<code>BoolLit</code>	'LOGICAL'
<code>Var</code>	tipo da symbol table
<code>ArithmeticBinOp</code>	'REAL' se algum operando for <code>REAL</code> ; caso contrário 'INTEGER'
<code>RelationalBinOp</code>	sempre 'LOGICAL'

Nó	Tipo devolvido
<code>LogicalBinOp</code>	sempre <code>'LOGICAL'</code>
<code>LogicalUnaryOp</code>	sempre <code>'LOGICAL'</code>
<code>ArithmeticUnaryOp</code>	tipo do operando
<code>FuncCall(MOD)</code>	<code>'INTEGER'</code>
<code>FuncCall(SQRT)</code>	<code>'REAL'</code>
<code>FuncCall(MAX/MIN)</code>	<code>'REAL'</code> se algum arg for <code>'REAL'</code> ; caso contrário <code>'INTEGER'</code>

Promoção numérica em expressões aritméticas

Em `_type_of_ArithmeticBinOp`, se um dos operandos for `REAL` e o outro `INTEGER`, o tipo resultante é `REAL`. Isto reflecte as regras de promoção do Fortran 77 (mixed-mode arithmetic).

Operadores relacionais com CHARACTER

A comparação relacional aceita `CHARACTER` além de tipos numéricos (para comparação lexicográfica), mas não permite misturar `CHARACTER` com tipos numéricos na mesma expressão.

Funções Intrínsecas Reconhecidas

```
_INTRINSICS: dict[str, tuple[int | None, str]] = {
    'MOD': (2, 'INTEGER'),
    'SQRT': (1, 'REAL'),
    'MAX': (None, 'REAL'),      # aridade variável, mínimo 2
    'MIN': (None, 'REAL'),      # aridade variável, mínimo 2
}
```

`MAX` e `MIN` com aridade `None` significam aridade variável. A verificação exige mínimo de 2 argumentos. O tipo de retorno é refinado em runtime — se todos os args forem `INTEGER`, devolve `INTEGER`; se algum for `REAL`, devolve `REAL`.

Acumulação de Erros vs. Falha Rápida

O `SemanticAnalyser` acumula todos os erros encontrados e devolve-os no final, em vez de lançar exceção ao primeiro erro. Esta decisão permite ao utilizador ver todos os problemas semânticos de uma vez, em vez de corrigi-los um a um. O método `analyse()` devolve `list[SemanticError]`; o chamador verifica `has_errors` antes de prosseguir para a tradução.

Limitações Conhecidas

- **Tipos implícitos Fortran 77:** o Fortran 77 tem regras de tipagem implícita (variáveis começadas por `I–N` são `INTEGER` por defeito, restantes são `REAL`). Esta implementação não suporta tipagem implícita

— todas as variáveis devem ser declaradas explicitamente.

- **Arrays unidimensionais apenas:** a gramática e o semantic só suportam arrays com uma dimensão.
- **Data-flow parcial:** a verificação de "variável não inicializada" só é feita para a variável de retorno de funções, não para variáveis gerais.
- **Parâmetros passados por referência:** o Fortran 77 passa todos os argumentos por referência. A implementação passa por valor (copia para a stack), o que é suficiente para a maioria dos casos mas não suporta modificação de variáveis do chamador através de parâmetros.